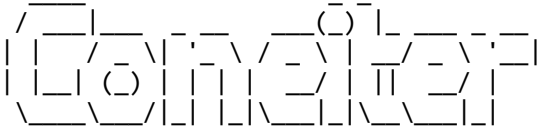


Time Complexity: Understanding Algorithm Efficiency

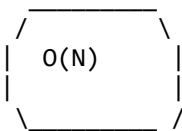
Your first step into Data Structures & Algorithms!



Introduction to Time Complexity

- **Time Complexity** is a fundamental concept in Computer Science.
- It measures the **time taken by an algorithm as a function of the length of its input (N)**.
- **Purpose:** To compare two or more algorithms and determine which one is more efficient, especially as input size grows.
- **Why not just run and time algorithms?**
 - Execution time depends on hardware, system load, and other factors, making direct comparison unreliable.
 - Time Complexity provides a standardized, hardware-agnostic way to evaluate performance.
- **Importance:** Frequently asked in interviews and crucial for competitive programming to choose the right algorithm and avoid Time Limit Exceeded (TLE) errors.

What is Big O Notation?



- The **Big O Notation** is used to quantify the time complexity of an algorithm.
- It describes the **upper bound** or **worst-case scenario** for an algorithm's running time.
- *"This algorithm will take at most this much time. It can take less, but it will surely take this much."*

Calculating Big O Complexity

When determining Big O notation, follow these rules:

- **Ignore Smaller Powers:** Only consider the term with the highest power of N.
- **Ignore Constants:** Disregard any constant factors associated with the highest power.

Examples:

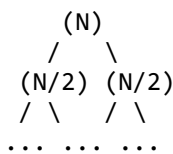
- $3n^2 + 5n$
 - Highest power: n^2
 - Big O: **$O(n^2)$**
- $n + 100 \log n$
 - Comparing n and $100 \log n$, n grows faster.
 - Big O: **$O(n)$**
- $3n^3 + 4n^5$
 - Highest power: n^5
 - Big O: **$O(n^5)$**
- 10000
 - No 'n' dependency; it's a constant number of operations.
 - Big O: **$O(1)$** (*Constant time complexity*)
- More **$O(n)$** examples:
 - $n/4$
 - $n + 100/4$
 - $5n + 3 \log n$ ($5n$ dominates $3 \log n$; ignore constant 5)
- More **$O(n^2)$** examples:
 - $n^2/1000$
 - $(n^2 + 10n)/5$

Other Notations (Brief Mention)

- **Theta Notation (Θ):** Describes the *exact bound* of an algorithm's running time (both upper and lower).
- **Omega Notation (Ω):** Describes the *lower bound* of an algorithm's running time.
- **Big O Notation** is most commonly used in Computer Science because it provides a safe upper estimate for algorithm performance.



Time Complexity of Recursive Functions



Finding the time complexity of recursive functions can be challenging, but the **Tree Method** simplifies it.

Tree Method Steps:

1. **Convert to T(n) Function:** Express the recursive function as a recurrence relation $T(n)$.
2. **Identify Non-Recursive Work:** Determine the work done at each level of the recursion (e.g., loops, constant operations).
3. **Draw the Recursion Tree:** Visualize the function calls as a tree.
4. **Calculate Work per Level:** Sum the non-recursive work at each level of the tree.
5. **Determine Tree Height:** Find out how many levels the tree has until the base case is reached.

6. **Calculate Total Work:** Multiply the work per level by the height of the tree, then simplify using Big O rules.

Example: $\text{fun}(n)$ calling $2 * \text{fun}(n/2)$ with a loop of N operations

Consider a function $\text{fun}(n)$ with:

- A base case (e.g., $n == 0$ returns).
- A for loop that runs n times (cost: $n * c$, where c is a constant for basic operations).
- Two recursive calls: $\text{fun}(n/2)$ and $\text{fun}(n/2)$.

1. Convert to $T(n)$ Function:

- $T(n) = 2 * T(n/2) + n*c$

2. & 3. Recursion Tree & Work per Level:

- **Level 0:** $n*c$
- **Level 1:** $T(n/2)$ and $T(n/2)$
 - Non-recursive work for each: $(n/2)*c$
 - Total work at Level 1: $(n/2)*c + (n/2)*c = n*c$
- **Level 2:** Four calls to $T(n/4)$
 - Non-recursive work for each: $(n/4)*c$
 - Total work at Level 2: $4 * (n/4)*c = n*c$
- **Pattern:** Each level performs $n*c$ operations.

4. Determine Tree Height:

- The input ' n ' is halved at each step: ' n ', ' $n/2$ ', ' $n/4$ ', ..., until it reaches ' 0 ' or ' 1 '.
- This pattern implies a logarithmic height: **$\log n$** (base 2).

5. Calculate Total Work & Big O:

- Total operations = (Work per level) * (Number of levels)
- Total operations = $(n*c) * (\log n)$
- Applying Big O rules (ignore constants): **$O(n \log n)$**

Practice Questions (Solutions in Telegram Channel)

Try finding the time complexity for these recursive functions:

- $T(n) = T(n/4) + T(n/2) + c$
- $T(n) = 2T(n-1) + c$



Online Judges & Constraints: The 10^8 Operations Rule



When solving problems on online judges (e.g., Codeforces, LeetCode), understanding constraints is key to choosing an efficient algorithm and avoiding **Time Limit Exceeded (TLE)** errors.

- **The Rule:** Generally, a program on an online judge server can perform approximately **10^8 operations per second**. If your algorithm exceeds this for the given time limit (usually 1-2 seconds), it will TLE.
- **Using Constraints:** The problem statement provides constraints on the input size (N). You can use N to estimate the acceptable time complexity.
- **Rough Guide for Time Complexity based on N :**
 - **$N < 10-11$:** Algorithms up to $O(N!)$ or $O(N^6)$ might be acceptable. (e.g., $11! < 10^8$, $11^6 < 10^8$)
 - **$N = 15-18$:** Algorithms like $O(2^N * N)$ or $O(N^4)$ might pass.
 - **$N = 100$:** Algorithms up to $O(N^4)$ might pass.
 - **$N = 400$:** Algorithms up to $O(N^3)$ might pass.
 - **$N = 1000$:** Algorithms up to $O(N^2)$ might pass.
 - **$N = 10^5$:** Algorithms like $O(N \log N)$ are expected. (e.g., sorting-based algorithms)
 - **$N = 10^6 - 10^7$:** Algorithms must be $O(N)$ or $O(\log N)$.
- By knowing the constraint on N , you can infer which type of algorithm (e.g., sorting, dynamic programming, brute force) is likely required.

✓ Conclusion & Further Practice

We've covered:

- What is **Time Complexity** and its importance.
- The fundamentals of **Big O Notation**, including how to calculate it and its significance as an upper bound.
- A systematic approach (**Tree Method**) to determine the time complexity of **recursive functions**.
- How to leverage **online judge constraints** and the **10^8 operations rule** to select appropriate algorithms.

Practice: Solve the Time Complexity questions available on InterviewBit (link in description) to solidify your understanding. Many are MCQ-based and will help you apply these concepts.